T6. Texturas

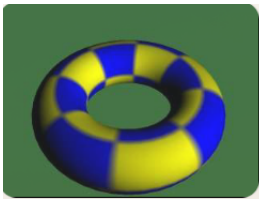
Introducción

Una **textura** es una colección o array n-dimensional de **valores** que se representan sobre la geometría de los objetos. La utilización de texturas: aumentan el **realismo** de la imagen sin incrementar el número de polígonos, e proporciona una clave para la **percepción** de la **profundidad**, permite simular multitud de efectos (sombras, entornos, reflejos...). Sen embargo, ocupan memoria.

Texturizado

El **texturizado** consiste en **superponer** sobre los polígonos **imágenes** que aumenten su realismo. Existen varias técnicas de texturizado:

- <u>Texture Mapping</u>

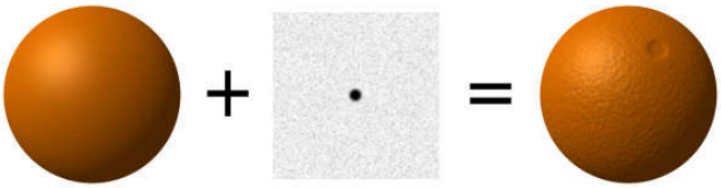


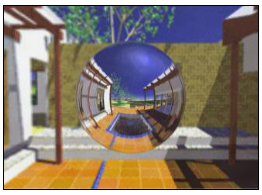
Crea una textura a partir de un patrón (imagen) para aplicarla sobre la superficie.

- <u>Bump Mapping</u>

Perturba la superficie mediante una modificación de las **normales** de las caras del objeto, pero **sin cambiar su geometría**. En sentido estricto puede no considerarse texturizado, es un tipo de **mapeado topológico**.

Las normales se calculan en función de un **mapa de alturas** (una matriz de datos $2^n * 2^n$, generalmente una imagen). Se combinan las normales con la geometría del objeto y se calcula el modelo de iluminación para esos nuevos valores de las normales.





- <u>Environment Mapping</u>

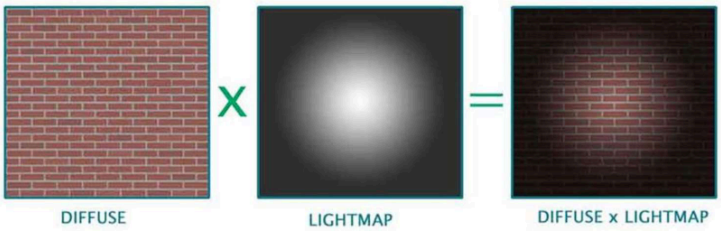
Mapea el **entorno** sobre la superficie.

- <u>Light mapping</u>

Mapea un **mapa de iluminación** sobre el objeto.

El brillo de las superficies en una escena se calcula previamente y se almacena en **mapas de textura** para aplicarse como un **mapa de iluminación** sobre la

superficie. Se suelen aplicar con mayor frecuencia a **objetos estáticos** para proporcionar efectos de iluminación como la iluminación global a un costo muy bajo.



- <u>Displacement mapping</u>

Modifica la geometría de un objeto mediante un mapa de texturas, generalmente cambiando la dirección de las **normales** de la misma. Esto genera una gran cantidad de **geometría adicional,** puede generar sombras, oclusiones y sombreados. Sin embargo, es lento.

Nota: **Geometry shader**, es un shader que se aplica después del vertex shader para generar geometría.

Mapeado de Texturas

- <u>Tipo texturas</u>

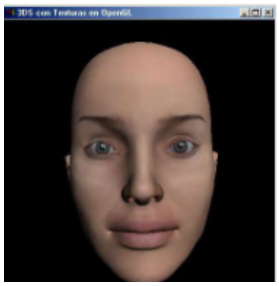
Es posible utilizar tres tipos.

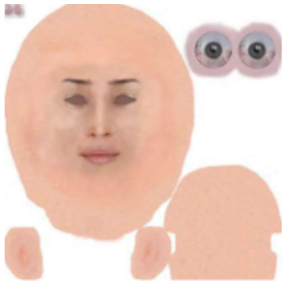
- **Texturas 1D**: son imágenes sin alto, sólo con ancho.
- **Texturas 2D**: son imágenes Bitmaps ordinarias.
- **Texturas 3D**: sólo las aceptan algunos sistemas, ocupan mucha memoria.

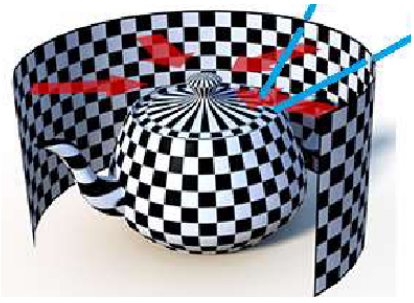
- <u>Aplicación</u>

El **mapeado de texturas** es la correspondencia de los puntos de la textura con los puntos de la superficie en la que se aplica. Tipos de mapeado:

- **Mapeado directo**: la textura se mapea directamente sobre el objeto.
- **Mapeado en dos fases**: la textura se mapea sobre un objeto 3D (esférico, cilíndrico u otros) y se proyecta sobre el objeto.



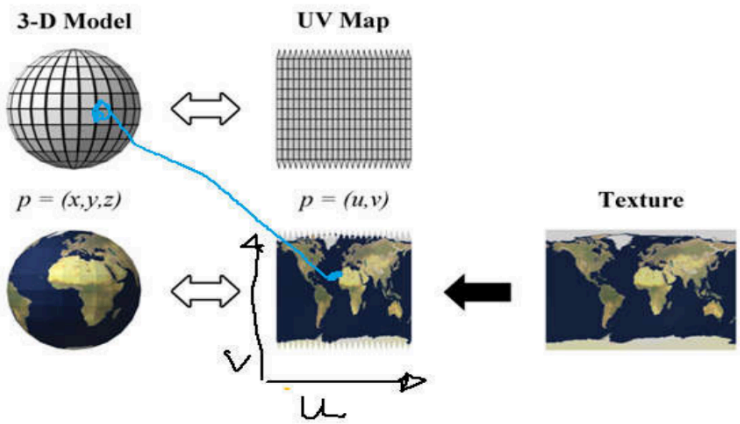


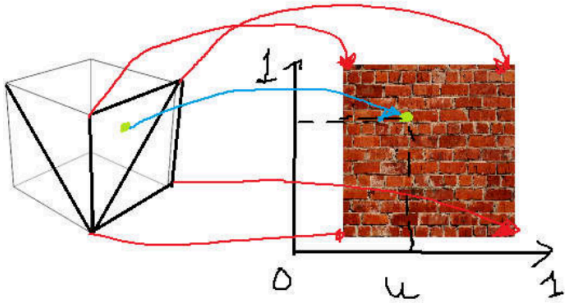


- <u>Mapeado Directo de Texturas (u, v)</u>

Para calcular el valor de un pixel determinado es necesario utilizar la **relación** entre la **textura** (u, v) y los **vértices** superficie. El objetivo es **obtener** las **coordenadas** de textura (u, v); unos valores que nos permitan indicar que coordenadas de la textura corresponden a cada vértice. En general este proceso es muy **complejo** y suele utilizar un programa de diseño CAD para obtener las coordenadas de texturas.

Las **coordenadas** (u, v) son un sistema de coordenadas utilizado para mapear texturas en superficies **3D**. Estas coordenadas definen cómo una imagen 2D (la textura) se "envuelve" o se proyecta sobre un objeto 3D. El **mapa de texturas** asocia un punto $T(u, v)$ con un punto (x, y, z) de la geometría del objeto.





- <u>Mapeado de Texturas en OpenGL</u>

Se siguen los siguientes pasos:

1. **Habilitar** el uso de **texturas** 1D, 2D o 3D.
2. **Nombrar** la textura (índices).
3. **Almacenar** en memoria, dando valores o leyendo un fichero.
4. **Definir** la textura y dar propiedades (describir el tipo de imagen).
5. **Mapear** la textura (describir como se colocan sobre el objeto).
6. **Activar** la textura (hacerla operativa) para cada objeto.

```
glBindTexture(GL_TEXTURE_2D, texture);
```

```
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aColor;
layout (location = 2) in vec2 aTexCoord;
out vec2 TexCoord;
out vec3 ourColor;
uniform mat4 transform;

void main() {
   gl_Position = transform * vec4(aPos, 1.0);
   ourColor = aColor;
   TexCoord = vec2(aTexCoord.x, aTexCoord.y);}
```

```
#version 330 core
in vec3 ourColor;
in vec2 TexCoord;
uniform sampler2D texture;

void main(){
gl_FragColor = texture(texture, TexCoord);
}
```

- <u>Habilitar el Mapeado de Texturas</u>

Para activar el uso de texturas, los objetos dibujados posteriormente usarán la textura asociada, en el main() o en la función $InitOpenGL$ se usa $glEnable(GL_TEXTURE_2D)$.

Si se quiere **deshabilitar** para que alguna superficie aparezca **sin textura**, con el color original se usan $glDisable(GL_TEXTURE_2D)$.

Para activar una **textura concreta**, se usa $glBindTexture(GL_TEXTURE_2D,\ Indice)$, $Indice$ es el identificador de la textura cargada previamente. Todo lo que se dibuje con $GL_TEXTURE_2D$ habilitado, usará esa textura.

Para **desvincular** cualquier textura se usa $glBindTexture(GL_TEXTURE_2D,\ 0)$.

- <u>Leer y Activar la Textura</u>

```
void glGenTextures( GLsizei n,  GLuint * textures );
```

- *n* Es el número de índices a crear.
- *textures* Un puntero al primer elemento del array en donde almacenarán los índices creados

```
int textura;
glGenTextures(1, &texture);
int arraytex[];
glGenTextures(n, &arraytex[]);
```

```
void glBindTexture( GLenum target, GLuint texture );
```

- *target* : GL_TEXTURE_1D o GL_TEXTURE_2D.
- *texture:* índice de la textura que queremos enlazar (no en uso)

- `glBindTexture(GL_TEXTURE_2D, texture[0]);`
- ***Esto sirve tanto para hacerla operativa como usarla después al vuelo como las listas o los VAO o VBO.***

- **`glBindTexture(GL_TEXTURE_2D, 0);  // no queremos usar ninguna`**

- **`glDeleteTextures (GLsizei num, const Gluint *nombres)`**

19

- <u>Almacenar en Memoria</u>

Una textura se puede especificar como un array de **texels**, que se puede crear manualmente.

```
GLubyte imageData1[3 * 256 * 256]
```

Un byte para cada color RGB.

ancho

Alto

```
unsigned char Colores[4][4][3] = {
   {{64,0,0},  {128,0,0},   {192,0,0},   {255,0,0}},  // Rojos
   {{0,64,0},  {0,128,0},   {0,192,0},   {0,255,0}},  // Verdes
   {{0,0,64},  {0,0,128},   {0,0,192},   {0,0,255}},  // Azules
   {{64,64,0}, {128,128,0}, {192,192,0}, {255,255,0}} // Amarillos
};
```

Normalmente, o bien se **escribe** un programa que la genere (texturas procedurales) o se lee de un fichero de imagen con la librería STB.

```
• //Librerías de cabecera para manejo de imagenes.
    #define STB_IMAGE_IMPLEMENTATION
    #include <stb_image.h>

int myCargaTexturas(char *nome){
int textura;
glGenTextures(1, &textura);
glBindTexture(GL_TEXTURE_2D, textura);
int width, height, nrChannels;                          //RBG, RGBA
unsigned char *data = stbi_load(nome, &width, &height, &nrChannels, 0);
.............
}
```

```
unsigned char *stbi_load(
  char const *filename,  // Ruta del archivo de imagen
  int *x,                // Ancho de la imagen (load)
  int *y,                // Alto de la imagen (load)
  int *channels_in_file, // Canales de color en la imagen (load)
  int desired_channels   // Canales deseados en la salida
```

Desired_channels:
Número de **canales deseados** en la salida.
- •0: Usar los canales originales de la imagen.
- •1: Forzar la imagen a escala de grises.
- •3: Forzar la imagen a RGB.
- •4: Forzar la imagen a RGBA.

En las **texturas procedurales**, la "imagen" es calculada a partir de una función matemática, se precisa un algoritmo, una función R^n. Estas texturas ocupan poco espacio, se pueden re-escalar fácilmente y no consumen memoria sino CPU. Es útil para superficies grandes o pequeñas.

- <u>Definir la Textura</u>

$void\ glTexImage2D(objetivo,\ nivel,\ componentes\ ,\ ancho\ ,\ alto,\ borde,\ formato,\ tipo,\ pixels);$

- **objetivo**: indica cuál es la textura a definir ($GL_TEXTURE_1D$ o $GL_TEXTURE_2D$).
- **nivel**: indica el nivel de detalle de las texturas, normalmente 0.
- **componentes**: indica el número de componentes de color usados en cada pixel.
- **ancho**: indica el ancho de la textura, que debe ser potencia de 2.
- **alto**: indica el alto de la textura, que debe ser potencia de 2.
- **borde**: indica el número de pixeles de borde que se usarán.
- **formato**: indica el tipo de valores esperados (normalmente GL_RGB).
- **tipo**: indica el tipo de la variable que almacena los valores de color por pixel.
- **pixeles**: puntero a la variable que almacena los colores de cada pixel.

```
glTexImage2D(GL_TEXTURE_2D, 0, 3, width2, height2, 0,GL_RGB,
                                GL_UNSIGNED_BYTE, imageData2);
```

```
// Es necesario para que el stb_image funcione
#define STB_IMAGE_IMPLEMENTATION
#include <stb_image.h>

int myCargaTexturas(char *nome){
            int textura;
            glGenTextures(1, &textura);
            glBindTexture(GL_TEXTURE_2D, textura);
// load image, create texture and generate mipmaps
            int width, height, nrChannels;
            unsigned char *data = stbi_load(nome, &width, &height, &nrChannels, 0);
            glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, data);
//Parametros
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
            glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

            stbi_image_free(data);
            return (textura);

•                                                    }
```

- <u>Pegar la Textura - Opengl 3.3</u>

La cargan los vértices y se especifican las coordenadas de textura.

```
float vertices[] = {
//Frontal vertices              Normales                        texturas
  -0.5f, -0.5f,  0.5f,          0.0f, 0.0f, 1.0f,               0.0, 0.0,
  0.5f, -0.5f,  0.5f,           0.0f, 0.0f, 1.0f,               1.0, 0.0,
  0.5f,  0.5f,  0.5f,           0.0f, 0.0f, 1.0f,               1.0, 1.0,
  0.5f,  0.5f,  0.5f,           0.0f, 0.0f, 1.0f,               1.0, 1.0,
  -0.5f,  0.5f,  0.5f,          0.0f, 0.0f, 1.0f,               0.1, 1.0,
  -0.5f, -0.5f, 0.5f,           0.0f, 0.0f, 1.0f,               0.0, 0.0,
…….
// vertices
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
// Normales
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);
// texture
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(6 * sizeof(float)));
glEnableVertexAttribArray(2);
```

- <u>Múltiples texturas</u>

Se leen dos texturas y se asignan distinto identificador (texture1 y texture2). Se modifica el shader para añadir ambos uniforms.

```
in vec2 TexCoord;
uniform sampler2D texture1;
uniform sampler2D texture2;
void main(){
     FragColor = mix(texture(texture1, TexCoord), texture(texture2, TexCoord), 0.3);
}
```

Se mapean las texturas sobre las variables uniformes.

```
glUseProgram(shaderProgram);
glUniform1i(glGetUniformLocation(shaderProgram, "texture1"), 0);
glUniform1i(glGetUniformLocation(shaderProgram, "texture2"), 1);
```

Y se usan.

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, texture);
glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, texture2);
```

```
glBindTexture(GL_TEXTURE_2D, texture);
```

```
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aColor;
layout (location = 2) in vec2 aTexCoord;
out vec2 TexCoord;
out vec3 ourColor;
uniform mat4 transform;

void main() {
   gl_Position = transform * vec4(aPos, 1.0);
   ourColor = aColor;
   TexCoord = vec2(aTexCoord.x, aTexCoord.y);}
```

```
#version 330 core
in vec3 ourColor;
in vec2 TexCoord;
uniform sampler2D texture;

void main(){
gl_FragColor = texture(texture, TexCoord);
}
```

En OpenGL 3.3 es posible y fácil utilizar más de una textura a la vez sobre un objeto. Se usan **unidades de textura** que permiten ir activando o desactivando las texturas manualmente.

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, texture1);
glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, texture2);
sphere.draw();
```

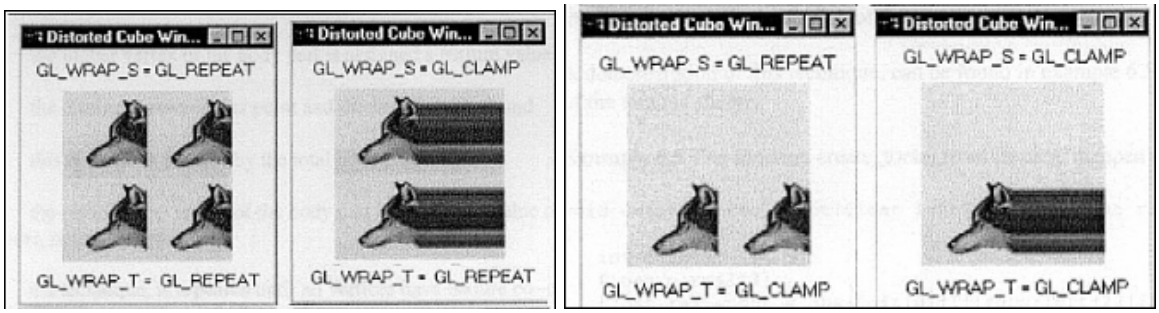
```
#version 330 core
out vec4 FragColor;
in vec3 ourColor;
in vec2 TexCoord;
// texture sampler
uniform sampler2D texture1;
uniform sampler2D texture2;

void main() {
FragColor = mix(texture(texture1, TexCoord), texture(texture2, TexCoord), 0.9);
}
```

Parámetros de Texturas

Sobre el **objeto**: son problemas de **tamaño**, qué hacer si la textura no abarca todo el objeto.

$glTextParameteri(GL_TEXTURE_2D,\ GL_TEXTURE_WRAP_S/T,\ GL_REPEAT/CLAMP)$

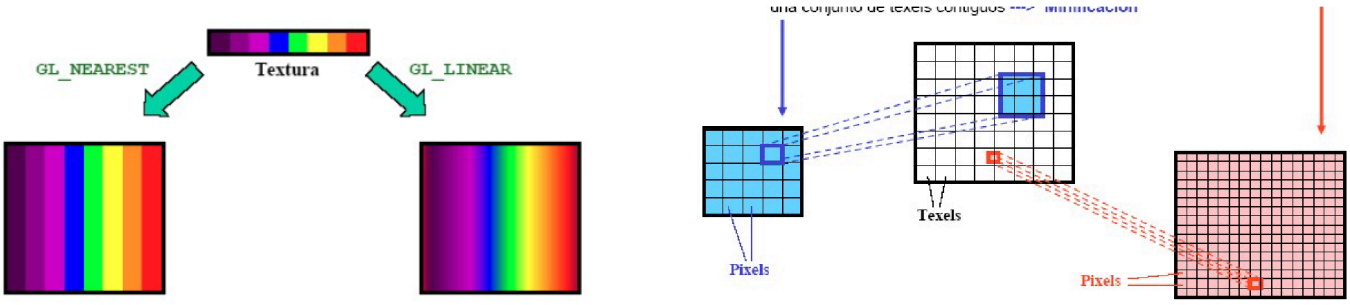
Especifica qué ocurre la si la textura no cubre los límites del objeto, puede repetirse (GL_REPEAT), o estirarse (GL_CLAMP). Debe llamarse dos veces, para la vertical (T) y para la horizontal (S).



Sobre la **pantalla**: problemas de **escala**, qué hacer si al rasterizar cada texel de la textura no se corresponde exactamente con un pixel de la pantalla.

$glTextParameti(GL_TEXTURE_2D,\ GL_TEXTURE_MIN/MAG_FILTER,\ GL_LINEAR/NEAREST)$

Si la superficie es mayor que la textura, cada pixel se corresponderá con un trozo pequeño de texel: **magnificación**. Si la superficie es menor que la textura, cada pixel se corresponderá con un conjunto de texels contiguos: **minificación**.



Mezclas: que hacer si el objeto tenía un color.

$glTextEnvi(GL_TEXTURE_ENV,\ GL_TEXTURE_ENV_MODE,\ GLint\ valor)$

Para establecer el modo RGBA donde $valor$ puede ser ($p = pixel$, $s = textura$, $p = entorno$):

- $GL_REPLACE$: $C_V = C_T$, **reemplaza** el color del objeto por la textura.

- $GL_MODULATE$: $C_V = C_S * C_T$, se **multiplica** el color del objeto y el de la textura.
- GL_DECAL: $C_V = C_T * C_\alpha$, sustituye teniendo en cuenta el canal Alpha.
- GL_BLEND: $C_V = C_P * (1 - C_S) + C_C * C_S$, multiplica el color del píxel por el color de la textura y del entorno un color elegido.

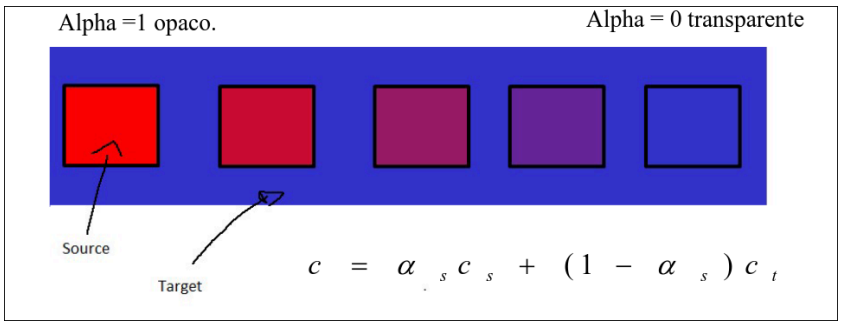
$glTextEnvfv(GL_TEXTURE,\ GL_TEXTURE_ENV_COLOR,\ GLfloat\ color[4])$ para enviar el color.

Transparencia

Blending-Mezcla: la forma más sencilla de hacer una **transparencia** es mediante la técnica de puerta o ventana transparente. Se usa una ecuación de la forma:

$$c = \alpha_S c_S + (1 - \alpha_S) c_t$$

siendo c_t el color objetivo y c_S el color fuente.



El modelo RGBA acepta el valor **alpha** (0.0 es totalmente transparente y 1.0 es opaco). Para ellos las texturas o color deben tener un alpha, por ejemplo .png o .gifa. Se habilita con $glEnable(GL_ALPHA_TEST)$.

El **alpha test** compara el valor del fragmento con un valor de referencia. Por defecto, este valor es el 0 y la comparación se hace entre 0 y 1.

$glAlphaFunc(GLenum\ func,\ GLclamp\ ref)$ donde $func$ puede ser GL_NEVER, GL_LESS, GL_EQUAL, GL_LEQUAL, $GL_GREATER$, $GL_NOTEQUAL$ o GL_ALWAYS (por defecto).

Para cualquier otro elemento de color RGBA:

$glEnable(GL_BLEND)$
$glBlendFunc(GL_SRC_ALPHA,\ GL_ONE_MINUS_SRC_ALPHA)$ donde GL_SRC_ALPHA usa el alfa del fragmento actual y $GL_ONE_MINUS_SRC_ALPHA$ calcula la contribución del fondo.

El orden de dibujo es: primero dibujar objetos opacos y después los transparentes, de **atrás hacia adelante**.